

Linguagens de Programação

Valores e Tipos de Dados

Bruno Lopes

Propriedades desejáveis

Legibilidade: A leitura do programa é facilmente compreendida?

Redigibilidade: A implementação reflete o algoritmo? A redação é sucinta?

Confiabilidade: É fácil detectar “enganos” do programador?

Eficiência: Roda rápido?

Facilidade de Aprendizado: É enxuta?

Ortogonalidade: Conceitos podem ser combinados livremente?

Reusabilidade: É possível aproveitar partes em outros programas?

Modificabilidade: É fácil alterar programas?

Portabilidade: Roda da forma esperada em diferentes plataformas?

Dado

Definição

Qualquer entidade que existe durante uma computação.

Pode ser:

- avaliado
- armazenado
- incorporado em estrutura de dados
- passado como argumento
- retornado em função

Dado

Definição

Qualquer entidade que existe durante uma computação.

Pode ser:

- avaliado
- armazenado
- incorporado em estrutura de dados
- passado como argumento
- retornado em função

Dado

Definição

Qualquer entidade que existe durante uma computação.

Pode ser:

- avaliado
- armazenado
- incorporado em estrutura de dados
- passado como argumento
- retornado em função

Tipos de dados

Pascal

- Números inteiros e reais
- caracteres
- sequências de caracteres
- valores lógicos
- registros
- arrays
- ponteiros
- ⋮

C

- Números inteiros e reais
- caracteres
- sequências de caracteres
- valores lógicos
- registros
- arrays
- ponteiros
- ⋮

Tipos podem ser simulados através de outros tipos!

Tipos de dados

Pascal

- Números inteiros e reais
- caracteres
- sequências de caracteres
- valores lógicos
- registros
- arrays
- ponteiros
- ⋮

C

- Números inteiros e reais
- caracteres
- sequências de caracteres
- valores lógicos
- registros
- arrays
- ponteiros
- ⋮

Tipos podem ser simulados através de outros tipos!

Tipos de dados

Pascal

- Números inteiros e reais
- caracteres
- sequências de caracteres
- valores lógicos
- registros
- arrays
- ponteiros
- ⋮

C

- Números inteiros e reais
- caracteres
- sequências de caracteres
- valores lógicos
- registros
- arrays
- ponteiros
- ⋮

Tipos podem ser simulados através de outros tipos!

Tipo

Definição

Um conjunto de valores.

Inteiros

$\{\dots, -2, -1, 0, 1, 2, \dots\}$

Reais

$\{\dots, -2.0, \dots, -1.3, \dots, 0.0, \dots, 1.7, \dots, 2.99, \dots\}$

Caracteres

$\{ 'a', \dots, 'z', 'A', \dots, 'Z', '0', \dots, '9', '$', ' ', '(', \dots \}$

???

$\{ 'a', \dots, 'z', 0.0, \dots, 1, 2, \dots \}$

Tipo

Definição

Um conjunto de valores.

Inteiros

$\{\dots, -2, -1, 0, 1, 2, \dots\}$

Reais

$\{\dots, -2.0, \dots, -1.3, \dots, 0.0, \dots, 1.7, \dots, 2.99, \dots\}$

Caracteres

$\{ 'a', \dots, 'z', 'A', \dots, 'Z', '0', \dots, '9', '$', ' ', '(', \dots \}$

???

$\{ 'a', \dots, 'z', 0.0, \dots, 1, 2, \dots \}$

Tipo

Definição

Um conjunto de valores.

Inteiros

$\{\dots, -2, -1, 0, 1, 2, \dots\}$

Reais

$\{\dots, -2.0, \dots, -1.3, \dots, 0.0, \dots, 1.7, \dots, 2.99, \dots\}$

Caracteres

$\{a', \dots, 'z', 'A', \dots, 'Z', '0', \dots, '9', '$', ', ', '(', \dots\}$

???

$\{a', \dots, 'z', 0.0, \dots, 1, 2, \dots\}$

Tipo

- Uma coleção de valores que têm alguma propriedade em comum
- Coleção de valores que exhibe comportamento uniforme nas operações associadas ao tipo.

Tipo

tipo = valores + operações

Tipo

- Uma coleção de valores que têm alguma propriedade em comum
- Coleção de valores que exhibe comportamento uniforme nas operações associadas ao tipo.

Tipo

tipo = valores + operações

Tipo

Tipo

tipo = valores + operações

$T1 = \langle C1, O1 \rangle$ e $T2 = \langle C2, O2 \rangle$

Se $C1 = C2$ e $O1 \neq O2$ então $T1 \neq T2$?

Se $C1 \neq C2$ e $O1 = O2$ então $T1 \neq T2$?

Tipo

Por que tipos?

Evitar paradoxos!

Paradoxo de Russel

$$R = \{x \mid x \notin x\}$$

$$R \in R?$$

Tipo

Por que tipos?

Evitar paradoxos!

Paradoxo de Russel

$$R = \{x \mid x \notin x\}$$

$R \in R?$

Tipo

Por que tipos?

Evitar paradoxos!

Paradoxo de Russel

$$R = \{x \mid x \notin x\}$$

$$R \in R?$$

Em computação

- Ajudam a estruturar a solução do problema
- Ajudam a compreender expressões indicando o conjunto de valores que podem ser denotados por ela
- Ajudam a detectar erros: operações aplicadas a valores impróprios
- Ajudam a gerar códigos eficientes: representação adequada de valores

Em computação

- Ajudam a estruturar a solução do problema
- Ajudam a compreender expressões indicando o conjunto de valores que podem ser denotados por ela
- Ajudam a detectar erros: operações aplicadas a valores impróprios
- Ajudam a gerar códigos eficientes: representação adequada de valores

Em computação

- Ajudam a estruturar a solução do problema
- Ajudam a compreender expressões indicando o conjunto de valores que podem ser denotados por ela
- Ajudam a detectar erros: operações aplicadas a valores impróprios
- Ajudam a gerar códigos eficientes: representação adequada de valores

Em computação

- Ajudam a estruturar a solução do problema
- Ajudam a compreender expressões indicando o conjunto de valores que podem ser denotados por ela
- Ajudam a detectar erros: operações aplicadas a valores impróprios
- Ajudam a gerar códigos eficientes: representação adequada de valores

Linguagens não-tipadas

Um único tipo

- LISP
- PERL
- SELF

```
(define a 1)
(define b 'casa')
(+ a b)
```

Linguagens não-tipadas

Um único tipo

- LISP
- PERL
- SELF

```
(define a 1)
(define b 'casa')
(+ a b)
```

Tipagem estática

Tempo de compilação

A informação a respeito do tipo está amarrada ao identificador, usualmente, pela declaração de variáveis.

```
var i: integer;  
    a: char;  
begin  
    readln(i);  
    readln(a);  
    writeln(i+a);  
end.
```

Tipagem estática

Tempo de compilação

A informação a respeito do tipo está amarrada ao identificador, usualmente, pela declaração de variáveis.

```
var i: integer;  
    a: char;  
begin  
    readln(i);  
    readln(a);  
    writeln(i+a);  
end.
```

Tipagem dinâmica

Tempo de compilação

Erros *eventualmente* detectados na execução. Todos os caminhos da execução devem ser testados para garantir a ausência de erros

a não definido

```
(if (= 1 1) 2 a)  
(if (= 1 1) a 2)
```

Tipagem dinâmica

Tempo de compilação

Erros *eventualmente* detectados na execução. Todos os caminhos da execução devem ser testados para garantir a ausência de erros

a não definido

```
(if (= 1 1) 2 a)
(if (= 1 1) a 2)
```

Tipagem dinâmica

```
(define segundo(lambda(l) (car (cdr l))))
```

```
(segundo '(1 2 3))
```

```
(segundo '('(1 2 3) (4 5 6)))
```

```
(segundo '("manga" "abacaxi" 5 6))
```

Tipagem dinâmica

```
(define segundo(lambda(l) (car (cdr l))))
```

```
(segundo '(1 2 3))  
(segundo '('(1 2 3) (4 5 6)))  
(segundo '("manga" "abacaxi" 5 6))
```

Tipagem

Tipagem Estática

- ✓ Eficiência
- ✓ Segurança
- ✗ Flexibilidade

Tipagem Dinâmica

- ✗ Eficiência
- ✗ Segurança
- ✓ Flexibilidade

Valor do tipo

Pertence ao conjunto de valores definido pelo tipo.

Expressão do tipo

O resultado da expressão pertence ao conjunto de valores definido pelo tipo.

Cardinalidade do tipo

Cardinalidade do conjunto de valores do tipo.

Valor do tipo

Pertence ao conjunto de valores definido pelo tipo.

Expressão do tipo

O resultado da expressão pertence ao conjunto de valores definido pelo tipo.

Cardinalidade do tipo

Cardinalidade do conjunto de valores do tipo.

Valor do tipo

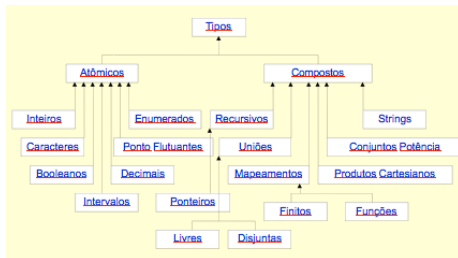
Pertence ao conjunto de valores definido pelo tipo.

Expressão do tipo

O resultado da expressão pertence ao conjunto de valores definido pelo tipo.

Cardinalidade do tipo

Cardinalidade do conjunto de valores do tipo.



Tipos Primitivos (Atômicos)

Seus valores não podem ser decompostos em valores mais simples!

O conjunto de tipos primitivos costuma revelar o propósito da Linguagem de Programação.

FORTRAN

- Inteiros
- Reais
- Complexos
- ⋮

Tipos Primitivos (Atômicos)

Seus valores não podem ser decompostos em valores mais simples!

O conjunto de tipos primitivos costuma revelar o propósito da Linguagem de Programação.

FORTRAN

- Inteiros
- Reais
- Complexos
- ⋮

Tipos Primitivos (Atômicos)

Sofrem limitações diretas do *hardware* e entre implementações da mesma linguagem.

Inteiros em Pascal

$[-32768, 32767]$

Inteiros em C

$[-65536, 65536]$

Tipos Primitivos (Atômicos)

Sofrem limitações diretas do *hardware* e entre implementações da mesma linguagem.

Inteiros em Pascal

$[-32768, 32767]$

Inteiros em C

$[-65536, 65536]$

Tipos Primitivos (Atômicos)

Built-in: pré-definidos na linguagem

Definidos: definidos pelo programador

Discretos: em correspondência direta com os inteiros